

Addis Ababa Science and Technology University

College of Engineering

Department of Software Engineering

Embedded Systems

**Smart Real-Time Driver State Monitoring
and Emergency Detection System
Project Proposal**

2025/26 Academic Year

II Semester



Group Members

No.	Name	ID/No
1	Tesfamichael Tafere	ETS 1303/15
2	Tewodros Gebretsadik	ETS 1305/15
3	Thressa Menilek	ETS 1309/15
4	Tigist Kinfemichael	ETS 1310/15
5	Tsegaye Berhe	ETS 1325/15

Submitted to Kassahun Tadesse

Submission date: March 2026

Table of Contents

1. Introduction	3
2. Problem Statement	4
3. Literature Review	4
3.1 Computer Vision for Drowsiness Detection	4
3.2 Accelerometer- and IoT-Based Crash Detection	5
3.3 Real-Time Edge Processing in Automotive Systems	6
3.4 Research Gap and Design Motivation	6
4. Project Objectives	6
5. Project Scope	7
5.1 In Scope	7
5.2 Out of Scope	7
6. System Overview and Merged Architecture	8
6.1 Two-Tier Safety Strategy	8
6.2 Hardware and Virtual Hardware Layer	9
6.3 Processing and Computer Vision Layer	9
6.4 Communication and Alerting Layer	9
6.5 Virtual Implementation Strategy	10
7. Functional and Non-Functional Requirements	12
Functional Requirements	12
Non-Functional Requirements	12
8. Methodology	12
9. Testing, Calibration, and Evaluation Plan	13
10. Required Resources	14
10.1 Virtual Hardware and Tool List	14
10.2 Software Libraries and Development Tools	14
10.3 Tool and Module Image Placeholders	15
11. Risk Analysis and Mitigation	16
12. Ethical, Privacy, and Safety Considerations	16
13. Expected Outcomes	17
14. Group Member Responsibilities	17
15. Project Timeline	18
16. Conclusion	18
17. References	19

1. Introduction

Road traffic injuries remain one of the most serious global public-health and transportation challenges. Beyond the direct loss of life, crashes produce substantial social and economic costs, especially when the event could have been prevented through earlier recognition of unsafe driving behavior or when emergency response is delayed. Driver fatigue, micro-sleeps, prolonged distraction, and impaired vigilance are especially dangerous because they undermine reaction time and situational awareness before the driver realizes the risk.

This proposal presents a Smart Real-Time Driver State Monitoring and Emergency Detection System that combines proactive accident prevention with post-impact emergency response in a single embedded design. The merged concept integrates computer-vision-based driver monitoring, threshold- and persistence-based risk classification, accelerometer-assisted crash recognition, and automated location-aware alert generation. In other words, the system is designed not only to warn the driver before a crash happens, but also to assist in obtaining help if a severe impact occurs.

Unlike many prototype projects that focus on only one safety function, the proposed system follows a two-tier safety strategy. The first tier continuously evaluates the driver's eye closure, blink behavior, yawning tendency, and head posture using computer vision techniques such as Eye Aspect Ratio (EAR) analysis and head-pose estimation. The second tier monitors simulated impact signals and, when a high-severity event is detected, prioritizes emergency communication logic that packages location information and prepares an SOS alert for designated contacts. This integrated perspective makes the project more meaningful academically and more relevant to real-world automotive safety challenges.

The final implementation approach for this course project is virtual rather than physical. Instead of constructing the hardware on breadboards, the team will develop a detailed virtual proof-of-concept using Proteus and related design tools, while implementing the detection algorithms and communication logic in software. This decision reduces procurement constraints, allows more emphasis on system design and algorithmic correctness, and enables a professional demonstration of end-to-end embedded behavior through simulation.

2. Problem Statement

Current driving environments suffer from multiple safety gaps that contribute to preventable fatalities. First, many vehicles—especially older or lower-cost vehicles—do not include internal sensing mechanisms capable of warning the driver before a dangerous fatigue or distraction episode develops into a crash. Second, once a collision occurs, timely emergency response may be delayed if the driver is injured, alone, driving at night, or traveling in a remote area. Third, commercial driver-monitoring technologies are often expensive, proprietary, and difficult to retrofit, making them inaccessible for many public, educational, or low-budget deployment contexts.

An additional design problem is the fragmentation of safety functions. In many cases, drowsiness detection and crash reporting are treated as separate solutions, leading to duplicated hardware, increased integration complexity, and higher cost. From an embedded-systems perspective, a more elegant solution is to unify sensing, decision-making, and alerting in one coherent architecture that can prioritize critical routines while still supporting rich monitoring logic. This is especially relevant for a software engineering project where architectural modularity, concurrency, and fail-safe behavior are important design concerns.

The proposed system therefore addresses four key needs: early recognition of risky driver states, cost-conscious integration of monitoring and emergency functions, reliable local alerting, and a scalable virtual prototype that demonstrates how embedded intelligence can be designed, validated, and documented even without physical hardware construction.

3. Literature Review

The literature relevant to this proposal spans three intersecting themes: computer vision for driver drowsiness detection, accelerometer- and IoT-based accident reporting, and edge or embedded architectures for automotive decision-making. Together, these studies support the decision to combine behavioral monitoring with emergency communication in one embedded design.

3.1 Computer Vision for Drowsiness Detection

Recent studies show that computer vision is one of the most practical ways to detect driver drowsiness without requiring intrusive physiological sensors. Reviews of driver drowsiness detection systems classify the field into image-based, biological, vehicle-based, and hybrid methods, and repeatedly note that image-based approaches are especially attractive for

deployable systems because they can monitor visible symptoms such as prolonged eye closure, blink changes, and yawning using inexpensive camera hardware. This supports a non-intrusive visual approach in the proposed system.

The literature also emphasizes the utility of facial-landmark-based methods such as Eye Aspect Ratio (EAR). In these approaches, the geometric relationship between vertical and horizontal eye landmarks is used to estimate whether the eyes are open or closed across consecutive frames. EAR can be paired with additional measures such as mouth opening and head posture to recognize micro-sleeps, distraction, or fatigue patterns more robustly than a single-feature method. Recent work further highlights that lightweight frameworks such as Dlib or MediaPipe, when combined with OpenCV, can achieve useful real-time performance on resource-constrained devices.

A further lesson from the literature is the importance of real-time robustness. Surveys and reviews note that high accuracy alone is not enough; systems must tolerate practical constraints such as low-light conditions, frame drops, and embedded-resource limits. This strengthens the case for using an IR-sensitive or low-light-capable camera in the conceptual design and for incorporating threshold calibration and persistence logic rather than relying on isolated frame events.

3.2 Accelerometer- and IoT-Based Crash Detection

Embedded accident-detection studies consistently use motion sensors, GPS modules, and GSM communication to shorten response time after a crash. A common approach is to employ an MPU6050 or a similar inertial sensor to track abrupt acceleration or orientation changes and compare them against calibrated thresholds. When the signal pattern indicates a likely crash, the controller retrieves location data from a GPS module and transmits an SMS alert through a GSM modem to predefined contacts or emergency responders.

This line of work is relevant to the proposed system for two reasons. First, it demonstrates that accelerometer-assisted crash detection can be built using low-cost embedded components. Second, it highlights that automated communication is especially valuable when the driver is unable to report the accident manually. For the present project, these ideas will be adapted into a simulated emergency-detection subsystem that models G-force spikes, GPS coordinates, SMS payload generation, and serial communication behavior within a virtual development environment.

3.3 Real-Time Edge Processing in Automotive Systems

Automotive safety applications increasingly rely on edge computing because safety decisions must be made with low latency and with reduced dependency on cloud connectivity. Reviews of edge and automotive architectures explain that modern vehicles are shifting toward architectures that concentrate substantial processing capability near the data source while still meeting functional safety and integration requirements. This supports the design choice of running driver monitoring logic locally rather than relying on remote inference.

The literature also explains that integrating multiple safety-critical functions requires careful architectural partitioning. Compute-intensive tasks such as video processing can interfere with time-critical sensing or interrupt handling unless the software architecture isolates responsibilities and establishes priority rules. This point is especially relevant for the present proposal: computer vision, crash detection, and communication logic should not be treated as one monolithic loop but rather as interacting modules with clear priority behavior and event-driven escalation.

3.4 Research Gap and Design Motivation

Although the literature demonstrates strong progress in both drowsiness detection and accident reporting, many academic prototypes still focus on only one subsystem. Fewer student-level proposals describe a merged architecture that unifies proactive monitoring and reactive emergency communication while also documenting how the software, virtual hardware, and testing strategy interact. Moreover, many published systems emphasize hardware assembly but provide less insight into design-for-simulation, verification workflows, and software engineering structure. This proposal fills that educational gap by presenting a complete, simulation-driven, embedded-system design that remains technically realistic while being feasible within a semester project.

4. Project Objectives

- To design and document a non-intrusive embedded system that monitors driver alertness through real-time computer vision.
- To implement a drowsiness-detection algorithm based on Eye Aspect Ratio (EAR), blink persistence, mouth opening, and head-posture cues using software tools such as OpenCV and Dlib or MediaPipe.

- To integrate a simulated MPU6050 crash-detection subsystem capable of recognizing sudden G-force spikes or abnormal motion events.
- To implement a communication subsystem that models GPS coordinate acquisition and GSM-based emergency SMS preparation after a severe impact.
- To develop the entire proof-of-concept in a virtual hardware environment using Proteus and associated software tools, avoiding the need for physical assembly while preserving embedded-system realism.
- To produce a professional academic report containing architecture diagrams, simulated circuit schematics, software design details, calibration strategy, demonstration evidence, and references.

5. Project Scope

5.1 In Scope

- Virtual hardware design of a dashboard-style monitoring and alerting system using Proteus or equivalent schematic/simulation tools.
- Software implementation of computer-vision-based driver monitoring using prerecorded video, live webcam input, or representative test clips.
- Simulation of the MPU6050, GPS, GSM, LCD, LEDs, and buzzer interactions using virtual peripherals and serial emulation tools.
- State-machine logic for integrating drowsiness detection, crash detection, priority handling, and emergency alert preparation.
- Calibration and validation of EAR thresholds, persistence windows, and crash thresholds using controlled simulation scenarios.
- Preparation of a complete academic demonstration package including figures, logs, pseudo-real user scenarios, and documented results.

5.2 Out of Scope

- Full commercial automotive deployment or automotive-grade certification.
- Autonomous intervention such as braking, steering control, or vehicle takeover.

- Medical diagnosis of sleep disorders or health conditions.
- A production-ready mobile application for long-term historical analytics.
- Physical breadboard or PCB construction as part of the semester deliverable.

6. System Overview and Merged Architecture

The merged architecture combines the broader academic structure of the original proposal with the stronger practical details from the revised concept. It is organized into layers so that the project remains easy to explain, modular to implement, and faithful to real embedded-system principles even in a virtual environment.

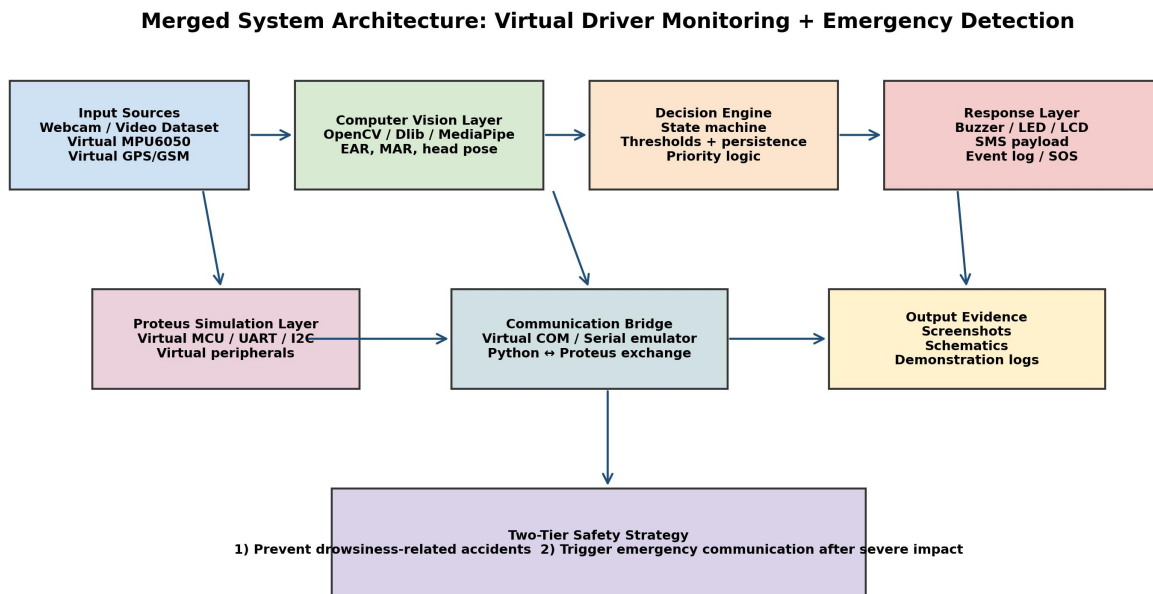


Figure 1. Merged architecture showing computer vision, virtual hardware simulation, communication bridge, and two-tier safety strategy.

6.1 Two-Tier Safety Strategy

The first safety tier is preventive. It continuously analyzes the driver's face, eye behavior, and posture to infer whether the driver is alert, distracted, or drowsy. When a warning-level condition is detected, the system activates local alarms such as LEDs, an LCD message, or an audible buzzer to encourage corrective action before an accident occurs. The second safety tier is reactive. If a severe impact is detected through the crash subsystem, the system prioritizes emergency

response logic, prepares an SOS message containing location information, and records the event for later demonstration and analysis.

6.2 Hardware and Virtual Hardware Layer

The conceptual hardware stack includes a processing unit (Raspberry Pi class board or ESP32-/Arduino-based virtual controller), a camera source, an MPU6050 inertial sensor, a GPS receiver such as the NEO-6M, a GSM modem such as the SIM800L, and local feedback components including an LCD, LEDs, and an active buzzer. In the actual course implementation, these components will be represented virtually rather than physically. Proteus will be used to build the schematic, connect the simulated controller and peripherals, and visualize the digital/serial behavior of the design.

This approach preserves the educational value of hardware-centric thinking. The team will still define pins, buses, power relationships, I2C and UART communication flows, and module interactions. However, instead of assembling wires on a breadboard, the connections will be tested and documented inside the virtual environment, allowing the project to focus on correctness, integration, and reproducibility.

6.3 Processing and Computer Vision Layer

The processing layer will operate outside or alongside the Proteus simulation using Python-based computer vision software. Video frames will be captured from a webcam or prerecorded dataset, facial landmarks will be extracted, and the Eye Aspect Ratio (EAR) will be computed over time. Head posture and yawning-related measures may be incorporated to improve robustness. If the EAR remains below a calibrated threshold across a sustained frame window, the software will assert a drowsiness state that can be forwarded to the simulated embedded system via a communication bridge or serialized event signal.

This design is especially suitable for a semester project because it separates high-level computer vision development from low-level virtual hardware behavior while still enabling end-to-end demonstration. It also supports repeated testing using identical recordings, which improves calibration quality and presentation consistency.

6.4 Communication and Alerting Layer

The communication and alerting layer integrates local and remote response. Local alerts include on-screen status, LED indicators, and a buzzer. Remote response includes GPS coordinate

retrieval, SOS message formatting, and GSM-oriented SMS logic. In the simulation, the GSM and GPS modules may be represented by virtual serial devices or emulators, allowing the project to demonstrate AT-command sequences, NMEA parsing logic, and emergency message construction even without real network transmission.

A key architectural principle is prioritization. In a severe crash scenario, emergency tasks should take precedence over noncritical processing so that the alerting path is not delayed by ongoing vision computations. In documentation and simulation logic, this will be represented through interrupt-aware or event-priority state handling rather than through a simplistic single-thread loop.

6.5 Virtual Implementation Strategy

Because the final deliverable is a virtual proof-of-concept, the project will be implemented as a hybrid simulation architecture. Proteus will host the circuit and virtual peripherals; Python/OpenCV will implement vision logic; and a bridge mechanism such as a virtual COM port or emulated serial interface will connect the detection outputs to the simulated controller behavior. This arrangement allows the team to demonstrate a realistic embedded workflow: visual analysis produces state events, the controller consumes them, and the virtual peripherals respond accordingly.

Virtual Development and Demonstration Workflow

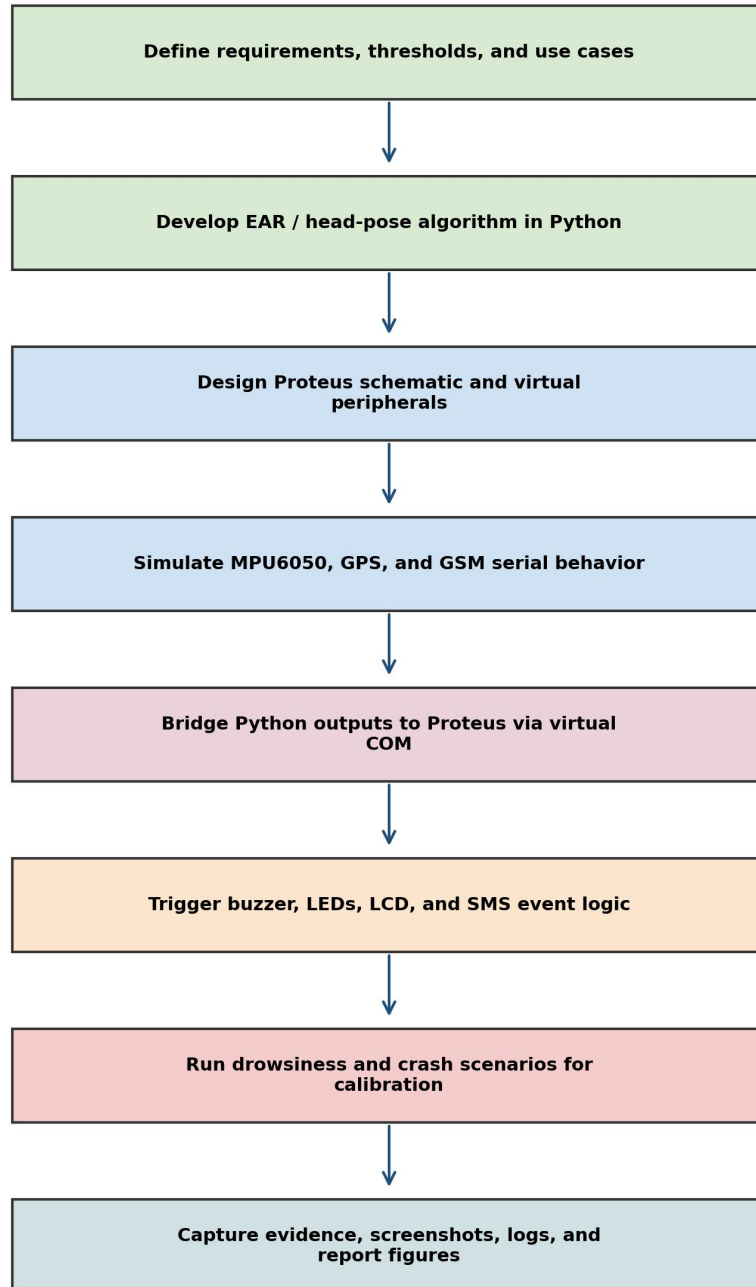


Figure 2. Virtual implementation workflow from algorithm development to simulation-based evidence capture.

7. Functional and Non-Functional Requirements

Functional Requirements

- The system shall analyze driver video frames in real time or pseudo-real time using computer vision algorithms.
- The system shall estimate driver drowsiness using EAR and related facial or postural cues.
- The system shall classify operational states such as normal, warning, alarm, and emergency.
- The system shall simulate crash events using MPU6050-like motion signals and detect abnormal impacts based on calibrated thresholds.
- The system shall generate local alarm responses through simulated LEDs, buzzer, and display output.
- The system shall generate a structured emergency message containing location information after a critical event.
- The system shall log or display system events for demonstration and verification purposes.

Non-Functional Requirements

- The solution shall remain modular so that computer vision, state logic, and virtual hardware can be developed in parallel.
- The response path for emergency events shall be treated as time-critical and must not be blocked by routine processing.
- The design shall be reproducible through software tools and documented clearly enough for another team to understand and simulate it.
- The system shall be cost-conscious in component selection and realistic as a future low-cost retrofit concept.
- The design shall account for privacy by preferring local processing and by avoiding unnecessary retention of raw driver video.

8. Methodology

The project will follow a six-phase methodology adapted to simulation-driven embedded development. In Phase 1, the team will define requirements, collect references, and prototype the

computer vision pipeline locally. This stage includes selecting the landmark framework, estimating initial EAR thresholds, and deciding how drowsiness and distraction events will be represented in software.

In Phase 2, the team will create the virtual hardware schematic in Proteus. The controller, LCD, LEDs, buzzer, and any modeled sensors or serial peripherals will be configured, and communication paths such as I2C, UART, or virtual COM connections will be mapped. This stage focuses on design completeness rather than physical assembly.

In Phase 3, the main software modules will be implemented. The computer vision algorithm will be refined in Python, the virtual controller logic will be written in C/C++ or an equivalent language, and emergency message generation logic will be created. In Phase 4, integration will begin: the vision outputs and the simulated hardware environment will be linked so that drowsiness or crash states can activate the virtual response system.

Phase 5 is dedicated to testing and calibration. The team will use controlled scenarios to reduce false positives caused by natural blinking, head movement, pothole-like motion, or sudden braking. Thresholds such as EAR, persistence duration, and crash-level G-force will be treated as experimentally tuned parameters rather than fixed assumptions. Phase 6 will finalize figures, circuit visuals, logs, screenshots, report writing, and demonstration material.

9. Testing, Calibration, and Evaluation Plan

- Algorithm testing using prerecorded videos or webcam sessions to observe EAR stability, blink detection, and head-pose behavior.
- Simulation testing of virtual buzzer, LED, LCD, GPS, and GSM behaviors inside Proteus.
- Integration testing through event transfer between the Python vision module and the Proteus environment.
- Crash-threshold testing using simulated G-force spikes to distinguish likely collisions from harmless motion disturbances.
- False-positive reduction through repeated threshold tuning and persistence-based triggering.
- Demonstration testing to ensure repeatable operation, screen readability, and clear evidence collection for the final presentation.

Evaluation will consider responsiveness, logical correctness, state-transition consistency, clarity of alerts, and evidence quality. Because the project is virtual, the evaluation emphasis will be on architectural completeness, simulation realism, and explainability rather than on claiming production-grade automotive performance. Where numerical thresholds are presented, they will be labeled as preliminary or calibration-dependent targets.

10. Required Resources

10.1 Virtual Hardware and Tool List

Tool / Module	Category	Purpose in Project	Note for Document Images
Proteus Design Suite	Simulation tool	Designs the virtual schematic and simulates controller-peripheral behavior.	Add official UI/tool screenshot later.
Arduino / ESP32 / compatible MCU model	Virtual hardware model	Acts as the simulated embedded controller in Proteus.	Insert a board image later.
Raspberry Pi or PC runtime	Processing platform	Runs the computer vision pipeline externally to the Proteus circuit.	Insert board or runtime image later.
USB webcam / video dataset	Input source	Provides facial video for EAR, yawning, and head-pose analysis.	Add camera or dataset image later.
MPU6050 model	Virtual sensor	Simulates crash-level motion and orientation events.	Add module image later.
NEO-6M GPS model	Virtual communication module	Provides simulated coordinates for emergency messages.	Add module image later.
SIM800L / serial GSM model	Virtual communication module	Models SMS transmission and AT-command workflow.	Add module image later.
LCD, LED, active buzzer	Virtual output peripherals	Shows and signals local alerts in the simulation.	Add real component images later.
Virtual COM / serial emulator	Integration utility	Bridges external software outputs with Proteus inputs or logs.	Add screenshot later.

10.2 Software Libraries and Development Tools

- Python: primary language for computer vision, experimentation, and event generation.
- OpenCV: image acquisition and frame-processing library for driver monitoring.
- Dlib or MediaPipe: facial landmark extraction framework for EAR and head-pose analysis.

- C/C++ (or Arduino-style firmware): virtual controller logic, state machine, and event handling.
- Proteus Design Suite: virtual schematic capture and embedded peripheral simulation.
- Fritzing or draw.io: additional circuit or architecture figure preparation.
- Git / GitHub: version control, collaboration, and code organization.
- Serial terminal / virtual COM tool: validation of SMS, GPS, and controller communication behavior.

10.3 Tool and Module Image Placeholders

To simplify later report polishing, Figure 3 provides a placeholder grid for the major tools and modules. The team can replace or supplement these placeholders with real images, screenshots, or official component photos before final submission.

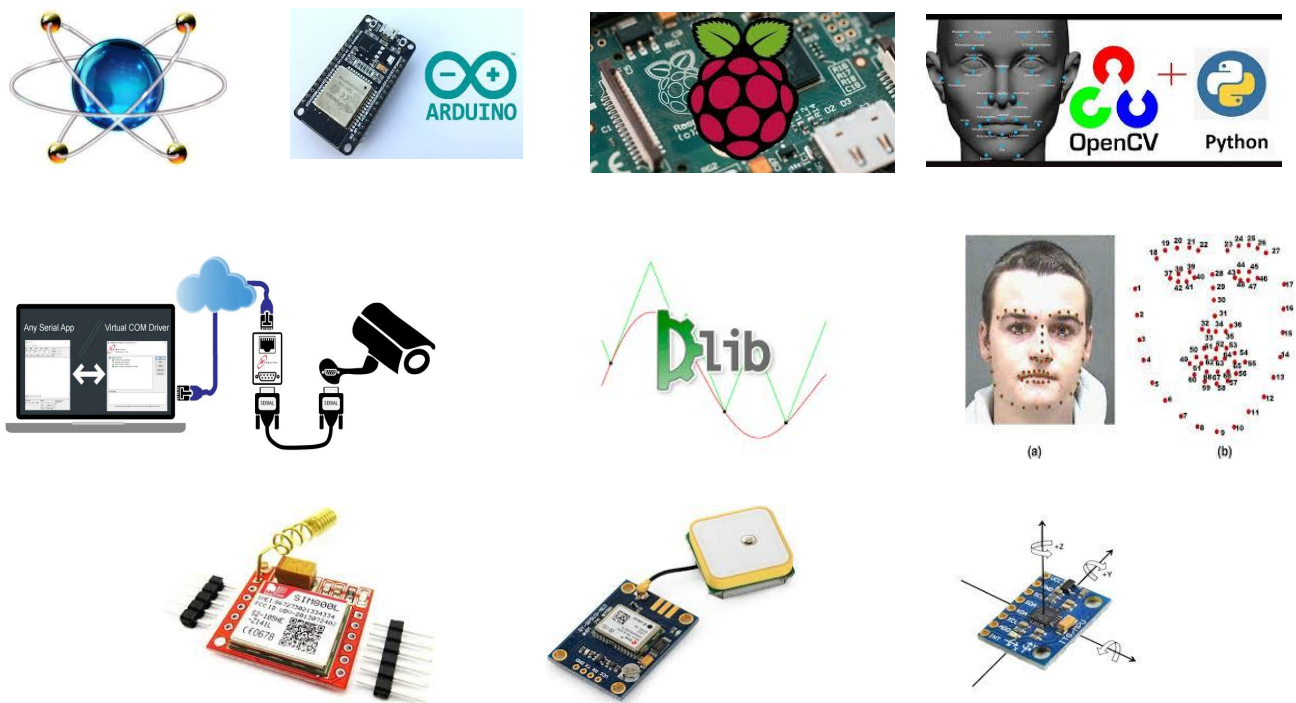


Figure 3. Real tools and module images.

11. Risk Analysis and Mitigation

Risk	Impact	Likelihood	Mitigation
False positives in drowsiness detection	Reduced trust in alerts	Medium	Use persistence windows, multiple features, and repeated threshold calibration.
Low-light or poor video quality	Unstable landmark detection	Medium	Use IR-capable conceptual design and test with varied input clips.
Weak integration between Proteus and external CV software	Incomplete end-to-end demo	Medium	Adopt a simple serial/event bridge early and test small increments.
Overly ambitious hardware realism in simulation	Time pressure and complexity	Medium	Prioritize core functions first, then extend communication realism.
Unverified threshold claims	Weak academic credibility	Medium	Present thresholds as preliminary and explicitly calibration-dependent.
Documentation gaps	Lower presentation quality	Low	Capture screenshots, logs, and schematics throughout development.

12. Ethical, Privacy, and Safety Considerations

Because the system monitors a human driver visually, privacy must be treated seriously even in a prototype. The proposal therefore prefers local or local-adjacent processing and does not depend on long-term cloud storage of raw driver video. Any demonstration footage should be collected with consent and used strictly for academic testing. Event logs should retain only the minimum information necessary to show system operation.

From a safety perspective, the system should be understood as an assistive technology rather than a guaranteed substitute for careful driving or certified emergency systems. The proposal aims to demonstrate a proactive and integrated embedded design, not to claim medical or automotive-grade certainty. Nevertheless, the design still benefits from responsible engineering practices such as prioritizing emergency routines, using calibration instead of arbitrary fixed values, and avoiding exaggerated performance claims.

13. Expected Outcomes

- A professionally documented, dashboard-oriented virtual prototype capable of detecting driver drowsiness through EAR and related visual cues.
- A simulated crash-detection subsystem using MPU6050-like signals and calibrated impact logic.
- A communication demonstration showing GPS coordinate parsing and GSM/SMS-oriented emergency payload generation.
- A clear local alert mechanism using virtual buzzer, LED, and display components.
- A complete academic package including architecture diagrams, virtual schematic evidence, screenshots, source-code structure, calibration notes, and presentation-ready figures.

14. Group Member Responsibilities

Because the team will not construct the hardware physically, responsibilities are aligned with software design, virtual hardware simulation, communication modeling, and integration testing. This structure allows each member to contribute deeply to a specific technical strand while preserving strong collaboration across the final proof-of-concept.

Member	ID/No	Key Responsibilities
Tesfamichael Tafere	ETS 1303/15	Technical Lead and Firmware Architect: Oversees the overall system state machine and software architecture. Develops the core C/C++ controller logic that manages interrupts, prioritizes emergency routines, and bridges computer vision outputs with the virtual hardware model.
Tewodros Gebretsadik	ETS 1305/15	Computer Vision and Algorithm Designer: Leads development of OpenCV and Dlib/MediaPipe scripts. Implements facial landmark detection, EAR calculation, and drowsiness logic using prerecorded datasets or live webcam feeds, and validates algorithm accuracy.
Thressa Menilek	ETS 1309/15	Hardware Simulation Engineer: Leads the virtual hardware design in Proteus. Designs the detailed circuit schematics, configures the virtual controller, and wires simulated LCD, LEDs, buzzer, and related peripherals inside the simulation environment.
Tigist Kinfemichael	ETS 1310/15	Communication Systems Designer: Develops the GPS and GSM communication logic. Scripts AT-command sequences for SMS delivery, handles NMEA parsing, and validates serial behavior using virtual terminals or emulator-based testing.

Tsegaye Berhe	ETS 1325/15	Sensor Simulation and Integration Testing Lead: Designs simulated MPU6050 motion inputs to mimic sudden G-force spikes. Leads end-to-end integration by linking external computer-vision software with Proteus through serial or virtual COM methods and documents system reliability.
---------------	-------------	---

15. Project Timeline

Phase	Activities	Deliverables
Phase 1: Requirement Analysis and Literature Review	Review driver monitoring and crash-detection literature; define functional requirements and preliminary thresholds.	Approved design scope and literature-backed concept.
Phase 2: Computer Vision Prototyping	Build EAR, yawning, and head-pose scripts on a PC using Python and OpenCV.	Validated algorithm prototype and baseline thresholds.
Phase 3: Proteus Hardware Simulation	Design virtual schematic, configure controller, outputs, and simulated serial peripherals.	Working Proteus circuit model.
Phase 4: Communication and Crash Logic	Implement virtual GPS/GSM logic, serial parsing, and crash-event simulation.	Emergency-message workflow and crash subsystem.
Phase 5: Integration and Calibration	Connect vision outputs to the Proteus simulation; tune thresholds and reduce false positives.	End-to-end virtual proof-of-concept with evidence.
Phase 6: Documentation and Demonstration	Prepare final report, annotated screenshots, architecture figures, and demo script.	Submission-ready proposal/report and live demonstration materials.

16. Conclusion

The updated Smart Real-Time Driver State Monitoring and Emergency Detection System proposal now reflects a stronger and more coherent concept. It combines the academic structure and broad software-engineering emphasis of the original document with the practical embedded details of the revised idea: EAR-based drowsiness detection, crash sensing through an MPU6050-style subsystem, GPS/GSM emergency alerting, and a clear two-tier safety philosophy. This merged design is more compelling because it frames the project not as a single-function detector but as a complete intelligent safety platform.

Equally important, the proposal is now aligned with the team's real execution strategy. By implementing the project virtually in Proteus and software tools rather than through physical hardware assembly, the team can focus on architecture, correctness, algorithm validation, integration logic, and professional documentation. The result is a feasible, well-scoped, and

academically rigorous embedded-systems project that is highly suitable for simulation-based demonstration and future extension.

17. References

- World Health Organization. (2023, December 13). Road traffic injuries.
<https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries/>
- National Highway Traffic Safety Administration. Drowsy driving: Avoid falling asleep behind the wheel. <https://www.nhtsa.gov/risky-driving/drowsy-driving>
- Albadawi, Y., Takruri, M., & Awad, M. (2022). A review of recent developments in driver drowsiness detection systems. *Sensors*, 22(5), 2069. <https://doi.org/10.3390/s22052069>
- Fu, B., Boutros, F., Lin, C.-T., & Damer, N. (2024). A survey on drowsiness detection—Modern applications and methods. *arXiv:2408.12990*.
- Kamboj, M., Kadian, K., Dwivedi, V., Wary, A., & Ojha, S. (2024). Advanced detection techniques for driver drowsiness: A comprehensive review of machine learning, deep learning, and physiological approaches. *Multimedia Tools and Applications*.
<https://doi.org/10.1007/s11042-024-19738-z>
- Hariharan, J., Varior, R. R., & Karunakaran, S. (2023). Real-time driver monitoring systems on edge AI device. *arXiv:2304.01555*. <https://doi.org/10.48550/arXiv.2304.01555>
- Almodhwahi, M. A., & Wang, B. (2025). A facial-expression-aware edge AI system for driver safety monitoring. *Sensors*, 25(21), 6670. <https://doi.org/10.3390/s25216670>
- Mat Desa, D. F. (2024). Vehicle accident alert system using GPS and GSM. *Progress in Computer and Mathematics Journal*, 1, 174–186.
- Askaripoor, H., Hashemi Farzaneh, M., & Knoll, A. (2022). E/E architecture synthesis: Challenges and technologies. *Electronics*, 11(4), 518.
<https://doi.org/10.3390/electronics11040518>
- Sawant, A. G., Kamble, S. S., Kanade, R. S., Kanugo, R. N., Kapse, T. A., & Bhapse, K. A. (2025). A real-time driver drowsiness detection system using MediaPipe and Eye Aspect Ratio. *arXiv:2511.13618*.